Computer Graphics (CSED451-01)
Assignment 1: 2D Drawing

Team Baguette - *Github Repository*

BLAIS Vladimir
CSED
49005916 - vblais

PICCAND Eliott
CSED
49005903 - piccandeliot

Development environment

To develop our program, we used VSCodium (with the clangd extension) for code editing, and CMake for compiling. However, instruction to compile with Visual Studio are available in the project `README.md`. To understand the source code, the reader must be familiar with modern C++ features (C++ 23).

In addition, we used the Tracy library to profile our game when we faced performances issues. This library was only used during the development and is not needed when compiling the game in Debug or Release mode.

Program design and implementation

Our program contains several features, including :

- basic player controls;
- enemies;
- collisions (with the world border, cannonballs and between ships);
- cannonballs aim, fire, travel and explosion;
- camera shaking on explosions;
- damage system;
- ship trail;
- water waves simulation;
- end (victory / defeat) menu.

and some features not visible by the players, but useful for development :

- hierarchical system;
- component system (inheritance based);
- event system;
- input system;
- UI auto layout.

In order to compile our program, there are 2 additional requirement[1]:

- enabling C++23 features. This choice was made to be able to use the C++23 `std::ranges` and `std::views` features, as well as the `<print>` header, to shorten development time and code readability.
- defining for the entire project the `GLM_ENABLE_EXPERIMENTAL` macro. This enable the use of the gtx glm extension, providing convenient functions such as 2D vectors rotations.

To summarize how our program works, we can use the following pseudo-code (see Algorithm 1)

Algorithm 1. – Game Main Loop

```
1:  function Main-Loop()
2:      while ¬ Window-Should-Close() do
3:          ▷ processes all events that occurred during the last
            frame
4:          Process-Events()
5:
6:          Process-Events()
7:          Update-Input-System()
8:          Update-UI-Events()
9:          ▷ update the different elements of the game, recur-
            sively from the scene root
10:         Update-Scene-Root()
11:
12:         ▷ render everything (this is a constant function, no
            update occurs here)
13:         Render-Scene-Root()
14:         ▷ more on this later
15:         Render-Hit-Point-Bars()
16:
17:         ▷ display the rendered content on the screen
18:         Swap-Framebuffers()
19:
20:         ▷ prepare every user input that occurred during the
            frame for the next update call
21:         Process-Inputs()
22:     end
23: end
```

This is a usual game main loop. Sometime, games place the `Process-Events()` part at the end of the loop, but we decided to place it at the beginning, since it has no impact on the behavior of the program, and allow to defer calls during the initialization (even if we do not need that so far, we might need it in the future).

We made the world square, but the program support any rectangle shape. This can be set by changing the `WORLD_WIDTH` and `WORLD_HEIGHT` constants inside `Utils/Constants.h`. Additionally, we added a margin around the work because we thought it looks nicer. This can be removed (such matching the exact assignment requirement) by setting the `WORLD_DISPLAY_MARGIN`[2] constant to `0.0f`.

This is how we implemented every player-visible feature :

[1] Please check the `README.md` for detailed instructions.

[2] located on line 37 of `Src/Components/Camera.cpp`

Basic player controls

To make the ship move we divided the work in several steps - all occurring during the ship entity update method. First, the program update the ship speed state and orientation based on the user keyboard using the input system. Then, it compute a unit vector indicating in which direction the ship is moving. Then, we add to the position of the ship this vector, multiplied by the ship's speed constant and the delta time[3]. Finally, it performs collisions checks.

Enemies

The game spawn 3 enemies at each game start. Those enemies have a basic AI : each enemy ship has a target position attached. This position is picked randomly among a list of hard-coded position, but excluding the ones that are too close to the current position. Once the position is picked, the enemy rotate to face the target, and since they are always moving forward, they end up getting closer to their target. Once they reached it, the whole process repeat.

To make the turret shoot cannonballs, we used the same behavior : the turret has a target that move exactly like the boat, but is invisible. Randomly, the turret shoot cannonballs at this target.

Collisions

Collisions are handled by the physics system (`RigidBody::simulateAll`) once per frame, after all the positions have been updated. For each pair of rigid bodies, we test overlap using SAT (Separating Axis Theorem) on their convex polygons.

When two ships overlap, we apply:

- a positional correction to separate them (to avoid interpenetration),
- then an impulse-based response (linear and angular), using mass, inertia, and restitution.

The solver runs several iterations each frame to improve stability.

Cannonballs are treated differently: they are kinematic (their motion is driven by their own update), so they do not receive impulse-based resolution. However, collisions involving cannonballs are still detected:

- cannonball vs ship posts a hit event, then damage is handled by the game logic,
- cannonball vs world/other object posts removal + explosion events,
- cannonball collisions with the shooter are ignored.

Finally, rigid bodies are clamped inside world bounds.

Cannonballs

Aim & Fire : During the `PlayerTurretControls` component's update method, the program check for mouse input using the input system. Depending on the buttons' states, the ship target position and a flag indicating whether the player is aiming are updated. When the player is aiming, a red arrow (see Fig. 2) indicate the position where the cannonball will explode (unless it collide with another ship before) and the turret is rotated to face that direction. Then, if the player is aiming and the left button is released, a `FireEvent` is sent to the event system, which will spawn an new cannonball entity on the next frame.

Travel : Each cannonball move the same way the ship does but user inputs cannot update its speed nor its direction. Cannonballs flight in a straight line, until they are close enough[4] to their target - or collided with anything but their shooter, at which point they trigger a `TargetReachedEvent` which delete the cannonball entity, and spawn a new explosion entity. Cannonballs are rendered as a simple yellow house shape (see Fig. 3). When firing, they inherit the shooter translation and rotation.

Explosion : Explosion update are quite simple : its radius is increased each frame, until the maximum is reached, then an `ExplosionDoneEvent` is sent, deleting the explosion entity.

The render part is a bit more complex : the program draw 3 layers, 3 times se same mesh, but with different scale, rotation and color. Layer's scale are determined by the current explosion radius : the first layer has that radius, and then, each layer scale is halved regarding the previous one. For their rotation, they are set randomly on the entity creation. Finally, their color is each different (first layer's color being red, last yellow and middle orange), but also varies with the explosion radius : at the beginning, each layer is white, then gradually blend with its own color, simulating an initial flash (see Fig. 4).

Camera Shake

On triggering the `TargetReachedEvent`, the camera start shaking. This is done by an algorithm like Algorithm 2

Algorithm 2. – Camera Shake

```
1:  function SHAKE()
2:      ▷ Crate a vector of length INTENSITY with a random
        orientation
3:      offset ← Random-Unit-Vector() × INTENSITY
4:
5:      while length(offset) > MIN_SHAKING do
6:          ▷ Flip the offset
7:          offset ← ROTATE(offset, 180°)
8:
9:          ▷ Slightly Rotate the offset by some random angle
10:         angle ← random(−60°, 60°)
```

[3]The delta time (`deltaTime` in the code), represent the duration in seconds of the previous frame. Multiplying the speed of moving parts of the game by this make the displayed speed independent of the current framerate, which vary from one computer to another.

[4]because position are floating point numbers and time steps are discrete trying to check if the cannonball reach the exact target position will always fail. Instead, the program check if the distance between the cannonball and the target is near 0.

```
    offset ← ROTATE(offset, angle)

    ▷ Decrease the offset intensity
    offset ← offset × INTENSITY_DECAY
  end
end
```

Damage System

As required by the assignment, we implemented a damage system. Each ship has a HP value, and when it reaches 0, the ship is sunk and deleted from the scene. To implement that, we used the event system.

When a `TargetReachedEvent` is triggered, the program check if the explosion is close enough to any ship (except the shooter) to damage it. If it is the case, a `DamageEvent` is sent, with the ship entity as target and the damage amount as data. When a ship receive a `DamageEvent`, its HP is decreased by the damage amount, and if it reaches 0, a `SinkEvent` is sent, deleting the ship entity.

To display the HP of the ships, we render a HP bar above each ship (see Fig. 1). The bar is green for the player and red for the enemies, and its length is proportional to the current HP of the ship.

Ship Trail

To display the ship foam trail (see Fig. 1), we decided to store the ship position at regular interval[5], and to display a point (`GL_POINTS` primitive) on each of those positions, with a different size and opacity depending on how long the position has been stored.

Because of drawing order issues, we had move the update and render part out of the Trail component, and defer it to a global Trail Renderer to ensure the trails keep updating once a skip is sunk and that the trail does not display over another ship.

Water Waves simulation

After all those features we still found the game looks flat, especially because of the water background. However, since we were not allowed to use textures nor shaders, we opted for a simulated background. We divided the world (100m × 1000m) into rectangles of 8m × 8m, associated a water height to each of these cell, and performed a simple simulation, inspired by the damped wave equation[6]. Thus the boat motion (see Fig. 1) and cannonballs (see Fig. 5) now interact dynamically with the surrounding water, creating waves and interferences patterns. However, adding this feature cost a lot of performances. This cost is not due to the simulation but by how we render the plane. Drawing a lot (15,625) rectangles with OpenGL immediate rendering result in a lot of draw calls. Thus, we switch to rendering this to a texture, and mapped the texture to a single rectangle.

Wooden Box Obstacles

We removed the wooden boxes present in the previous version of the game because it causes issues with the enemies AI and would have required too much development time to implement an avoidance system.

End Game Menu

When every enemy - or the player - has been sunk, the game pauses and display a victory / defeat menu. For this menu, we build a basic UI system with buttons, labels and containers, to display the game end status (victory / defeat) and give the user the choice to restart a new game or quit.

Font Rendering : Font rendering is quite hard to implement, and since we were not allowed to use external libraries, we had to build our own font rendering. For that, we used a trick : we wrote a Python script that loaded a font and drew every character onto a flat image (using PPM P3 format for easy parsing). In addition, the script generated a map from every character to its location on the generated image[7]. With that data now available onto the C++ part, whenever we need to draw some text, the program simply iterate over the text's characters, and copy-past the glyph from the texture atlas to the text texture. This method is not really suitable for a real scale game, but avoid having to load files at runtime.

Other Small Features

Ship Radars : Every ship has a radar attached to it. It is made of an opaque disc and a red transparent rotating pie slice. This is purely visual and has no impact on the gameplay.

Ship Flags : Every ship has a flag attached to it, the flag has a constant flapping animation. This is purely visual and has no impact on the gameplay.

End-user guide

The game starts immediately on running the executable. The player (blue ship) can change its ship speed with the `W` and `S` keys (respectively increasing and decreasing the boat speed), and rotate it using the `A` and `D` keys (turing the boat respectively left and right by 15°). In addition, the player can shoot cannonballs with his mouse : pressing the left click enable aiming mode which displays a ray toward the target. In aiming mode, 2 actions ar possible : cancel fire by clicking (press and release) the right click, or fire by releasing the left click. Fullscreen can be toggle by clicking the F11 key.

The game ends when every enemies (green ships) is sunk (Victory) or the player is sunk (Defeat) On game ends, the game pauses and a menu open, displaying the end status and 2 buttons to either restart or quit.

[5] We implemented that using a cyclic queue data structure - since there is only a limited amount of position needed each frame - to avoid allocating memory each frame.

[6] Our implementation is not the real discrete damped wave equation simulation, but a simplified version aiming to recreate its global behavior without diving into complex mathematics and physics.

[7] In the end, the Python script generated the `Include > Utils > Font > FontAtlas.h`.

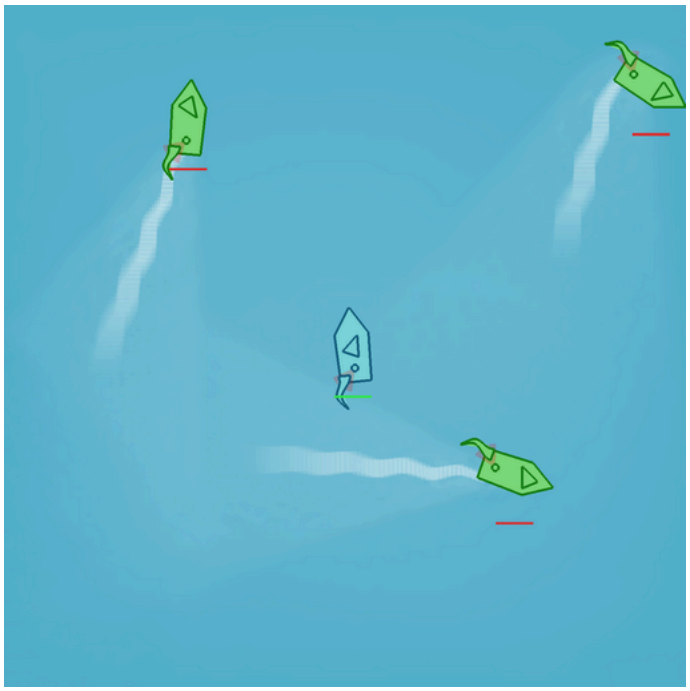

Fig. 1. – Game few seconds after the start. Are visible :

- the player ship (in blue in the middle) with its hp bar (in green, below);
- the enemies ships (in green in the corners) with their hp bar (in red, below);
- the foam trail behind moving enemy ships;
- a slight paler blue cone behind the enemies, created by the water waves simulation.

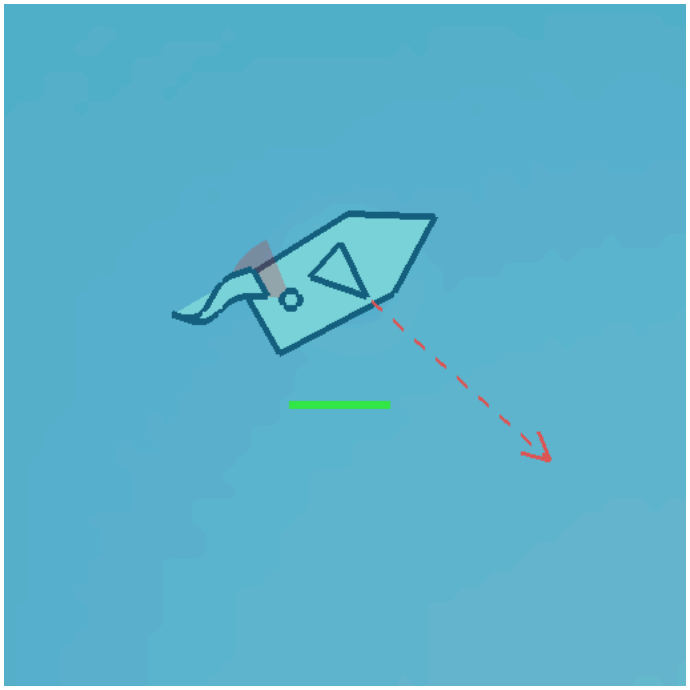

Fig. 2. – Player ship aiming

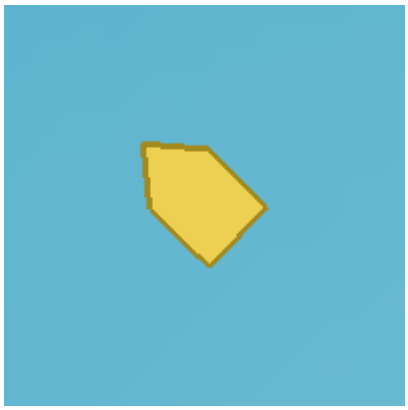

Fig. 3. – A cannonball

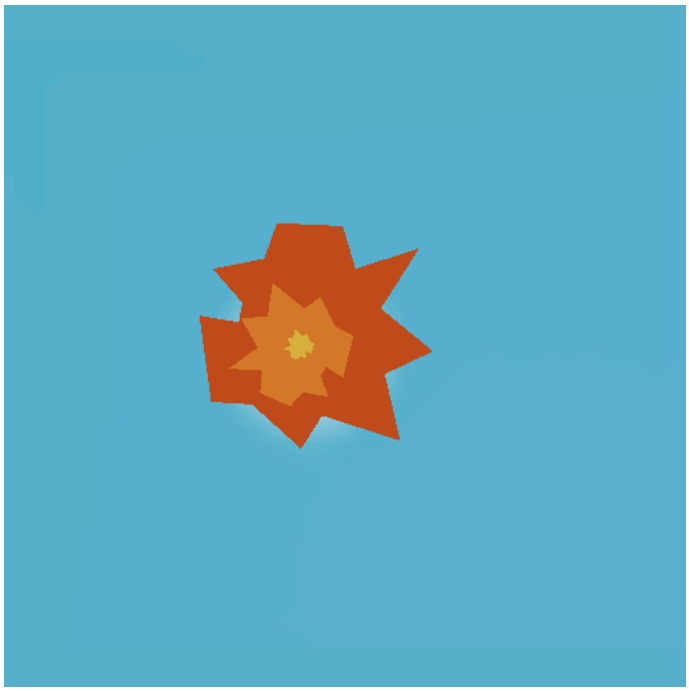

Fig. 4. – Final stage of the cannonball's explosion animation

Fig. 5. – Waves in the water after a cannonball explosion

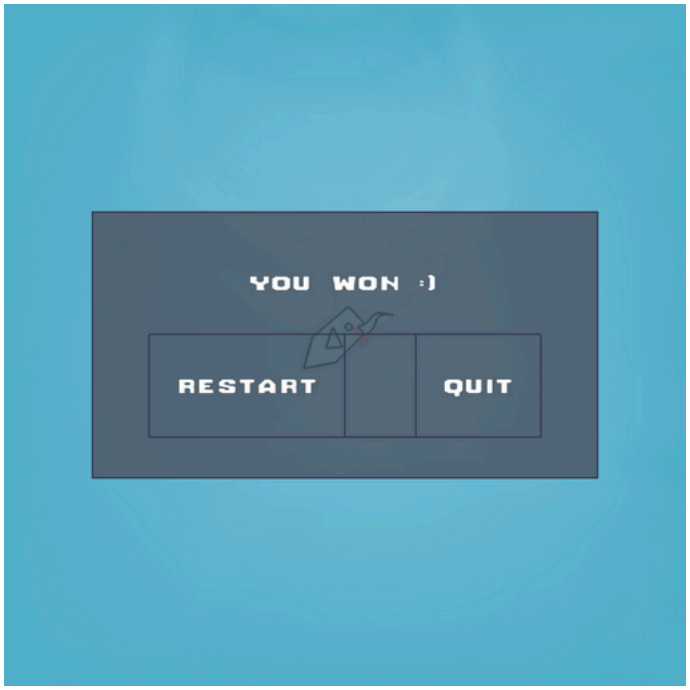

Fig. 6. – End game menu (if the game end with a player victory)

Discussions/Conclusions

During the development, we didn't encountered much issues. Creating the hierarchical game objects and component system was very long, but not hard. Drawing the font also required some shenanigans to work without any other library.

References

Every part of the code is original, but we use several tutorials or references during the development :

- the camera shaking part mechanic is greatly inspired by @miklatov answer on this Stack Exchange discussion;
- the water simulation part was inspired by the introduction of this post by Scott Lembcke;
- the collision system was implemented following this tutorial by timw;
- the component system was inspired by this Vulkan tutorial.

AI-assisted coding references

During the development, we used generative AI (Microsoft Copilot and Proton Lumo) for some parts :

- to make the custom `CyclicQueue` iterable. So `CyclicQueue::Iterator`, `CyclicQueue::begin()` and `CyclicQueue::end()` were generated using an LLM;
- Copilot helped implementing the collision system;
- Lumo helped update the water simulation rendering to a texture;
- Lumo helped find some bugs about circular smart pointers;
- The Python script generating the font atlas was also entirely made by Lumo.

Thus, we estimate than 2% of the code was generated by an LLM.